

Technical Assessment Task

Mini ERP Order & Inventory Management System

Objective

Build a backend system that manages products, inventory, sales orders, and accounting entries while handling concurrent requests safely.

Scenario

A company has multiple products and warehouses. Users should be able to:

- Create products
- Manage inventory
- Create sales orders
- Prevent overselling
- Generate accounting entries
- Get real-time stock updates
- Handle failed operations safely

1. Authentication & RBAC

Implement authentication using JWT.

Roles:

- Admin — Full access
- Manager — Manage products and inventory; view orders
- Sales User — Create orders; view products

2. Product Management

Create APIs for:

```
POST    /products
GET     /products
GET     /products/:id
PUT     /products/:id
DELETE  /products/:id
```

Product fields:

```
id
name
sku
price
createdAt
updatedAt
```

3. Inventory Management

Create inventory for products.

Fields:

```
productId
warehouseId
availableQuantity
reservedQuantity
updatedAt
```

APIs:

```
POST /inventory/add-stock
GET  /inventory/:productId
POST /inventory/adjust
```

Every inventory change should maintain a transaction history.

```
InventoryTransaction

id
productId
type: IN | OUT | ADJUSTMENT
quantity
referenceId
createdAt
```

4. Sales Order

Create an API:

```
POST /orders
```

Example request:

```
{
  "customerId": "123",
  "items": [
    {
      "productId": "abc",
      "quantity": 2
    }
  ]
}
```

Order Flow:

```
Create Order
  ↓
Validate Product
  ↓
Check Inventory
  ↓
Reserve/Deduct Stock
  ↓
Create Order
  ↓
Create Accounting Entry
  ↓
Send Real-Time Notification
```

5. Critical Concurrency Challenge

Initial stock:

```
Product A = 5 units
```

Two users simultaneously send:

```
Order 1 → Quantity 5
Order 2 → Quantity 5
```

Requirement: Only one order should succeed. Stock should never become negative.

Candidate must explain:

- How concurrency is handled
- Database transaction strategy
- Locking strategy OR atomic update
- What happens when one request fails

6. Accounting Entry

When an order is successfully created, create accounting entries.

Customer Receivable	Debit	■1000
Sales Revenue	Credit	■1000

Requirement:

Total Debit = Total Credit

Create tables/models for:

```
JournalEntry
id
referenceId
date
description
```

```
JournalEntryLine
journalEntryId
account
debit
credit
```

Order creation and accounting creation should maintain consistency.

If accounting creation fails:

```
Order → Rollback
Inventory → Rollback
```

Alternatively, the candidate should clearly justify an event-driven/distributed architecture.

7. Redis Caching

Implement Redis caching for either:

```
GET /products
OR
GET /inventory/:productId
```

Candidate must implement:

- Cache key strategy
- TTL
- Cache invalidation

Example:

```
inventory:product:{productId}
```

When inventory changes:

```
Database Update
↓
Invalidate Redis Cache
```

8. Real-Time Notification

Use Socket.IO / WebSocket.

When inventory changes:

```
Stock Updated
↓
WebSocket Event
↓
Connected Users Receive Update
```

Example event:

```
inventory_updated
```

Payload:

```
{
  "productId": "123",
  "availableQuantity": 25
}
```

9. Failed Operation Handling

Handle this scenario:

```
Order Created
↓
Inventory Updated
↓
Accounting Entry Failed
```

The system must not remain in an inconsistent state.

Possible approaches:

- Database Transaction
- Rollback
- Event-driven architecture
- Retry mechanism
- Compensation logic

Candidate should explain the chosen approach.

10. Bulk Import Challenge

Create an API:

```
POST /products/import
```

Upload CSV / Excel with:

```
Product Name
SKU
```

Price
Opening Stock

Requirements:

- Validate rows
- Detect duplicate SKU
- Return invalid row numbers
- Process large files efficiently

Bonus: Handle 100,000+ rows without blocking the main Node.js server.

11. Database Optimization Challenge

Create API:

`GET /orders`

Support:

- Pagination
- Date Filter
- Customer Filter
- Status Filter

`GET /orders?page=1&limit=20&status=COMPLETED`

Candidate should:

- Add proper indexes
- Avoid unnecessary queries
- Explain indexing decisions
- Provide EXPLAIN output for at least one important query

12. API Rate Limiting

Implement rate limiting.

`100 requests / minute / user`

Candidate can use:

- Redis
- Middleware
- Token bucket
- Fixed/sliding window

Recommended Tech Stack

- Node.js
- Express.js / NestJS
- TypeScript
- MySQL / PostgreSQL

- Redis
- Socket.IO
- Docker

Required Submission

1. GitHub Repository

Include proper commits.

2. README

- Project Setup
- Architecture
- Database Schema
- API Documentation
- Concurrency Handling
- Caching Strategy
- Assumptions

3. Docker Setup

Project should run using:

```
docker-compose up
```

Services:

- Backend
- Database
- Redis

4. API Collection

Provide either Postman Collection or Swagger/OpenAPI.

Post-Submission Interview Discussion

Question: Walk me through the complete order creation flow from API request to database commit.

Expected Answer: Should explain validation, transaction handling, stock validation, order creation, accounting entry, commit, cache invalidation, and event emission.

Question: Where exactly did you use a database transaction?

Expected Answer: Around operations that must succeed or fail together, especially inventory, order, and accounting changes.

Question: How did you prevent two simultaneous requests from overselling inventory?

Expected Answer: Should explain row locking, optimistic locking, or atomic conditional updates.

Question: Why did you choose your specific Redis cache strategy?

Expected Answer: Should justify key design, TTL, invalidation, and freshness requirements.

Question: What happens if Redis goes down?

Expected Answer: Application should gracefully fall back to the database while cache-dependent performance degrades.

Question: Why do you emit WebSocket events only after database success?

Expected Answer: To avoid notifying users about changes that later fail or rollback.

Question: How would you scale this system to 100 companies and 50,000 users?

Expected Answer: Discuss stateless API scaling, database optimization, caching, queues, load balancing, and tenant isolation.

Evaluation Criteria

Backend Architecture — 15%

Database Design — 15%

Concurrency Handling — 20%

Code Quality — 10%

Transaction Handling — 10%

Redis Implementation — 10%

API Design — 5%

Error Handling — 5%

Docker / Deployment — 5%

Documentation — 5%

Total — 100%